DAA Lab Codes (Simplified)

1. Selection Sort & Insertion Sort

```
#include <stdio.h>

void selectionSort(int a[], int n) {
   for(int i=0;i<n-1;i++) {
      int min=i;
      for(int j=i+1;j<n;j++)
         if(a[j]<a[min]) min=j;
      int t=a[i]; a[i]=a[min]; a[min]=t;
   }
}

void insertionSort(int a[], int n) {
   for(int i=1;i<n;i++) {
      int key=a[i], j=i-1;
      while(j>=0 && a[j]>key) {
         a[j+1]=a[j];
         j--;
      }
      a[j+1]=key;
   }
}

int main(){
   int a[]={5,3,2,4,1}, n=5;
   selectionSort(a,n);
   insertionSort(a,n);
}
```

2. Quick Sort

```
#include <stdio.h>

int partition(int a[], int l, int h) {
   int p=a[h], i=l-1;
   for(int j=l;j<h;j++) {
      if(a[j]<p) {
         i++;
         int t=a[i]; a[i]=a[j]; a[j]=t;
```

```
        }
    }
    int t=a[i+1]; a[i+1]=a[h]; a[h]=t;
    return i+1;
}

void quickSort(int a[], int l, int h) {
    if(l<h) {
        int pi=partition(a,l,h);
        quickSort(a,l,pi-1);
        quickSort(a,pi+1,h);
    }
}
```

3. Merge Sort

```
#include <stdio.h>

void merge(int a[], int l, int m, int h){
    int i=l,j=m+1,k=0,temp[100];
    while(i<=m && j<=h)
        temp[k++]=(a[i]<a[j])?a[i++]:a[j++];
    while(i<=m) temp[k++]=a[i++];
    while(j<=h) temp[k++]=a[j++];
    for(i=l,k=0;i<=h;i++,k++)
        a[i]=temp[k];
}

void mergeSort(int a[], int l, int h){
    if(l<h){
        int m=(l+h)/2;
        mergeSort(a,l,m);
        mergeSort(a,m+1,h);
        merge(a,l,m,h);
    }
}
```

4. Fractional Knapsack

```
#include <stdio.h>

int main(){
    int w[]={10,20,30}, v[]={60,100,120};
```

```
    float total=0;
    int cap=50;

    for(int i=0;i<3;i++){
        if(cap>=w[i]){
            total+=v[i];
            cap-=w[i];
        } else {
            total+=v[i]*(float)cap/w[i];
            break;
        }
    }
    printf("Max value = %.2f",total);
}
```

5. Dijkstra

```
#include <stdio.h>
#define V 4
#define INF 9999

int main(){
    int g[V][V]={
        {0,1,4,0},
        {0,0,2,6},
        {0,0,0,3},
        {0,0,0,0}
    };

    int dist[V], vis[V]={0};

    for(int i=0;i<V;i++) dist[i]=INF;
    dist[0]=0;

    for(int i=0;i<V;i++){
        int u=-1,min=INF;
        for(int j=0;j<V;j++)
            if(!vis[j] && dist[j]<min)
                min=dist[j],u=j;

        vis[u]=1;

        for(int v=0;v<V;v++)
```

```
        if(g[u][v] && dist[u]+g[u][v]<dist[v])
            dist[v]=dist[u]+g[u][v];
    }

    for(int i=0;i<V;i++)
        printf("%d ",dist[i]);
}
```

6. Floyd-Warshall

```
#include <stdio.h>
#define V 4
#define INF 9999

int main(){
    int g[V][V]={
        {0,3,INF,7},
        {INF,0,1,5},
        {INF,INF,0,INF},
        {INF,2,3,0}
    };

    for(int k=0;k<V;k++)
        for(int i=0;i<V;i++)
            for(int j=0;j<V;j++)
                if(g[i][k]+g[k][j]<g[i][j])
                    g[i][j]=g[i][k]+g[k][j];

    for(int i=0;i<V;i++){
        for(int j=0;j<V;j++)
            printf("%d ",g[i][j]);
        printf("\n");
    }
}
```

7. 0/1 Knapsack

```
#include <stdio.h>

int main(){
    int wt[]={1,3,4,5}, val[]={1,4,5,7};
    int dp[5][8]={0};
```

```
   for(int i=1;i<=4;i++){
      for(int w=1;w<=7;w++){
         if(wt[i-1]<=w)
            dp[i][w]= (val[i-1]+dp[i-1][w-wt[i-1]] > dp[i-1][w])
                  ? val[i-1]+dp[i-1][w-wt[i-1]]
                  : dp[i-1][w];
         else
            dp[i][w]=dp[i-1][w];
      }
   }

   printf("Max value = %d",dp[4][7]);
}
```

8. LCS

```
#include <stdio.h>
#include <string.h>

int main(){
   char a[]="ABC", b[]="AC";
   int m=strlen(a), n=strlen(b);
   int dp[10][10]={0};

   for(int i=1;i<=m;i++){
      for(int j=1;j<=n;j++){
         if(a[i-1]==b[j-1])
            dp[i][j]=dp[i-1][j-1]+1;
         else
            dp[i][j]= (dp[i-1][j]>dp[i][j-1]) ? dp[i-1][j] : dp[i][j-1];
      }
   }

   printf("LCS = %d",dp[m][n]);
}
```

9. N-Queens

```
#include <stdio.h>
#include <stdlib.h>

int n=4, x[10];
```

```
int place(int k,int i){
   for(int j=1;j<k;j++)
      if(x[j]==i || abs(x[j]-i)==abs(j-k))
         return 0;
   return 1;
}

void nqueen(int k){
   for(int i=1;i<=n;i++){
      if(place(k,i)){
         x[k]=i;
         if(k==n){
            for(int j=1;j<=n;j++)
               printf("%d ",x[j]);
            printf("\n");
         } else
            nqueen(k+1);
      }
   }
}

int main(){
   nqueen(1);
}
```